

## PROPUESTA

# Lenguaje de Dominio Específico para Deep Learning

*Paradigma Funcional | ANTLR v4 | Patrón Visitor en Python*

Primera Entrega: Gramática, Léxico y Ciclos

Integrantes: Mariana Duque; Lina Castañeda; Nicolás Sánchez; Jhonatan Valdés



## 1. Introducción

Este documento presenta el estado actual de la primera entrega del Lenguaje de Dominio Específico (DSL) RISCO, orientado a operaciones de deep learning y machine learning. El lenguaje sigue el paradigma de programación funcional e incorpora ciclos imperativos controlados (for y while), implementado usando ANTLR v4 con el patrón de diseño Visitor en Python.

Esta entrega cubre lo efectivamente implementado y funcional en el repositorio: gramática completa en ANTLR, léxico, tipos de datos básicos, operadores, variables, ciclos for/while/if y la función print().

### Filosofía de Diseño

El lenguaje está diseñado para ser minimalista, expresivo y matemáticamente elegante.

| Propuesta de Diseño de Lenguaje                                                        | DSL para Deep Learning |
|----------------------------------------------------------------------------------------|------------------------|
| Inspirado en Haskell (guards, composición) y Scala (val/var), con paradigma funcional. |                        |
| Todas las librerías internas serán escritas desde cero por el equipo de desarrollo.    |                        |
| Ejecución mediante consola o archivos .rc interpretados por ANTLR v4 + Python.         |                        |

## 2. Decisiones Generales de Diseño

| Característica    | Decisión                         | Justificación                                     |
|-------------------|----------------------------------|---------------------------------------------------|
| Paradigma         | Funcional con ciclos controlados | Limpio, matemático y componible                   |
| Tipado            | Dinámico y fuerte                | Sintaxis flexible, sin conversiones implícitas    |
| Inmutabilidad     | val (inmutable) / var (mutable)  | Funcional por defecto, mutable cuando se necesita |
| Cierre de bloques | Palabra clave end                | Claro y consistente en todas las estructuras      |
| Comentarios       | // una línea                     | Todavía no implementado anidado                   |
| Sistema de tipos  | Num, Decimal, Bool, Text, Null   | Tipos básicos implementados y funcionales         |
| Extensión         | .rc                              | Identifica archivos para el intérprete            |

## 3. Tipos de Datos

Los siguientes tipos de datos están implementados y reconocidos por el léxico y el evaluador del intérprete:

### 3.1 Tipos Básicos Implementados

| Tipo    | Descripción            | Ejemplo         |
|---------|------------------------|-----------------|
| Num     | Número entero          | val x = 5       |
| Decimal | Número decimal (Float) | val y = 3.14    |
| Bool    | Valor booleano         | val flag = true |
| Text    | Cadena de caracteres   | val s = "hola"  |
| Null    | Ausencia de valor      | val n = null    |

## 4. Palabras Reservadas

Las siguientes palabras reservadas están definidas en la gramática ANTLR y reconocidas por el lexer:

### 4.1 Control y Estructura

| Palabra | Categoría | Función                        |
|---------|-----------|--------------------------------|
| val     | Variable  | Declara una variable inmutable |

| Propuesta de Diseño de Lenguaje |             | DSL para Deep Learning                        |
|---------------------------------|-------------|-----------------------------------------------|
| var                             | Variable    | Declara una variable mutable                  |
| if                              | Condicional | Inicia un bloque condicional                  |
| elif                            | Condicional | Condición alternativa                         |
| else                            | Condicional | Caso por defecto del if                       |
| for                             | Ciclo       | Inicia un ciclo for                           |
| while                           | Ciclo       | Inicia un ciclo mientras                      |
| in                              | Ciclo / Op. | Itera en for / verifica pertenencia en listas |
| end                             | Bloque      | Cierra cualquier bloque (if, for, while)      |
| true                            | Literal     | Valor booleano verdadero                      |
| false                           | Literal     | Valor booleano falso                          |
| null                            | Literal     | Valor nulo                                    |
| print                           | Built-in    | Imprime un valor en consola                   |

## 4.2 Función Built-in: print()

La única función built-in implementada en esta entrega es print(). Está disponible globalmente sin necesidad de importar ningún módulo y es implementada directamente por el intérprete en Python.

| Función  | Signatura   | Descripción                                                    |
|----------|-------------|----------------------------------------------------------------|
| print(x) | Any -> Null | Imprime el valor x en la consola seguido de un salto de línea. |

Ejemplo de uso:

|                                  |
|----------------------------------|
| <code>print("Hola RISCO")</code> |
| <code>print(42)</code>           |
| <code>print(3.14)</code>         |
| <code>print(true)</code>         |

# 5. Operadores y Símbolos

## 5.1 Operadores Aritméticos

| Símbolo | Operación      | Ejemplo       |
|---------|----------------|---------------|
| +       | Suma           | val z = x + y |
| -       | Resta          | val z = x - y |
| *       | Multiplicación | val z = x * y |
| /       | División       | val z = x / y |
| ^       | Potenciación   | val z = x ^ 2 |
| %       | Módulo         | val z = x % 2 |

## 5.2 Operadores de Comparación y Lógicos

| Símbolo | Operación      | Ejemplo              |
|---------|----------------|----------------------|
| ==      | Igual          | if x == 5:           |
| !=      | Diferente      | if x != 0:           |
| >       | Mayor que      | if x > 0:            |
| <       | Menor que      | if x < 10:           |
| >=      | Mayor o igual  | if x >= 0:           |
| <=      | Menor o igual  | if x <= 100:         |
| &&      | Y lógico (AND) | if x > 0 && x < 10:  |
|         | O lógico (OR)  | if x < 0    x > 100: |
| !       | Negación (NOT) | if !flag:            |
| in      | Pertenencia    | if 3 in lista:       |

## 5.3 Operador in

El operador in verifica si un valor pertenece a una lista o si un carácter está en un string. Retorna Bool.

|                                      |         |
|--------------------------------------|---------|
| 2 in [1, 2, 3]                       | // true |
| "pera" in ["manzana", "pera", "uva"] | // true |
| "a" in "hola"                        | // true |

## 5.4 Símbolos de Comentarios

| Símbolo | Tipo                | Descripción                                  |
|---------|---------------------|----------------------------------------------|
| //      | Línea única         | Comenta el resto de la línea                 |
| ///     | Documentación       | Documenta una función (para IDE y auto-docs) |
| /-      | Apertura multilinea | Abre comentario multilinea (soporta anidado) |
| -/      | Cierre multilinea   | Cierra comentario multilinea                 |

*Nota: /- -/ soporta anidado porque el lexer cuenta los niveles de apertura y cierre, similar a los paréntesis.*

# 6. Sintaxis del Lenguaje

## 6.1 Variables

|                                      |
|--------------------------------------|
| // Inmutable (no se puede reasignar) |
| val x = 5                            |
| val nombre = "deep learning"         |
| val activo = true                    |
|                                      |

| Propuesta de Diseño de Lenguaje     | DSL para Deep Learning |
|-------------------------------------|------------------------|
| // Mutable (se puede reasignar)     |                        |
| var contador = 0                    |                        |
| var perdida = 100.0                 |                        |
|                                     |                        |
| // Reasignación de variable mutable |                        |
| var contador = contador + 1         |                        |

## 6.2 Listas

---

Las listas pueden contener valores de cualquier tipo. Se puede verificar pertenencia con `in` e iterar con `for`.

|                                         |
|-----------------------------------------|
| val numeros = [1, 2, 3, 4, 5]           |
| val frutas = ["manzana", "pera", "uva"] |
| val mixta = [1, true, 3.14]             |
| val vacia = []                          |

## 6.3 Condicionales

---

|                          |
|--------------------------|
| if promedio > 5:         |
| print("Promedio alto")   |
| elif promedio == 5:      |
| print("Promedio exacto") |
| else:                    |
| print("Promedio bajo")   |
| end                      |

## 6.4 Ciclo For

---

|                              |
|------------------------------|
| // For iterando una lista    |
| for item in [1, 2, 3, 4, 5]: |
| print(item)                  |
| end                          |
|                              |
| // For iterando un string    |
| for letra in "hola":         |
| print(letra)                 |
| end                          |

## 6.5 Ciclo While

---

|              |
|--------------|
| var i = 0    |
| while i < 5: |

| Propuesta de Diseño de Lenguaje | DSL para Deep Learning |
|---------------------------------|------------------------|
| <code>print(i)</code>           |                        |
| <code>var i = i + 1</code>      |                        |
| <code>end</code>                |                        |

## 6.6 Comentarios

|                                                                     |
|---------------------------------------------------------------------|
| <code>// Comentario de una línea</code>                             |
| <code>val x = 5 // También al final de una línea</code>             |
| <code>/// Comentario de documentación (para IDE y auto-docs)</code> |
| <code>/-</code>                                                     |
| <code>Comentario multilínea</code>                                  |
| <code>/-</code>                                                     |
| <code>El anidado está permitido sin conflictos</code>               |
| <code>-/</code>                                                     |
| <code>Esto sigue siendo comentario</code>                           |
| <code>-/</code>                                                     |

## 7. Programa de Ejemplo Completo

El siguiente programa demuestra todas las características implementadas en esta entrega. Los archivos RISCO tienen extensión `.rc`

|                                                                                 |
|---------------------------------------------------------------------------------|
| <code>// ejemplo_completo.rc</code>                                             |
| <code>// Demuestra: variables, listas, operadores, if, for, while, print</code> |
| <code>val datos = [10, 25, 3, 47, 8, 19, 33, 5]</code>                          |
| <code>var suma = 0</code>                                                       |
| <code>var mayor = 0</code>                                                      |
| <code>// Ciclo while: sumar todos los elementos y encontrar el mayor</code>     |
| <code>var i = 0</code>                                                          |
| <code>while i &lt; 8:</code>                                                    |
| <code>var suma = suma + datos[i]</code>                                         |
| <code>if datos[i] &gt; mayor:</code>                                            |
| <code>var mayor = datos[i]</code>                                               |
| <code>end</code>                                                                |
| <code>var i = i + 1</code>                                                      |
| <code>end</code>                                                                |
| <code>print("Suma total:")</code>                                               |
| <code>print(suma)</code>                                                        |

| Propuesta de Diseño de Lenguaje                 | DSL para Deep Learning |
|-------------------------------------------------|------------------------|
| print("Valor mayor:")                           |                        |
| print(mayor)                                    |                        |
|                                                 |                        |
| // Ciclo for: imprimir elementos que superen 10 |                        |
| print("Valores mayores que 10:")                |                        |
| for n in datos:                                 |                        |
| if n > 10:                                      |                        |
| print(n)                                        |                        |
| end                                             |                        |
| end                                             |                        |
|                                                 |                        |
| // Operador in: verificar pertenencia           |                        |
| val buscar = 47                                 |                        |
| if buscar in datos:                             |                        |
| print("El valor está en la lista")              |                        |
| else:                                           |                        |
| print("El valor no está en la lista")           |                        |
| end                                             |                        |
|                                                 |                        |
| // Operadores lógicos y aritméticos             |                        |
| val a = 10                                      |                        |
| val b = 20                                      |                        |
| if a > 5 && b < 30:                             |                        |
| print("Ambas condiciones se cumplen")           |                        |
| end                                             |                        |
|                                                 |                        |
| val potencia = 2 ^ 8                            |                        |
| val modulo = 17 % 5                             |                        |
| print(potencia)                                 |                        |
| print(modulo)                                   |                        |

## 9. Estructura del Proyecto

El repositorio está organizado de la siguiente manera:

|                 |                                       |
|-----------------|---------------------------------------|
| Risco-V1/       |                                       |
| gramaticas/     |                                       |
| RISCO.g4        | # Gramática del lenguaje (ANTLR v4)   |
| RISCOLexer.py   | # Generado por ANTLR                  |
| RISCOParser.py  | # Generado por ANTLR                  |
| RISCOVisitor.py | # Generado por ANTLR (patrón Visitor) |
| src/            |                                       |
| risco.py        | # Intérprete principal                |

| Propuesta de Diseño de Lenguaje | DSL para Deep Learning                    |
|---------------------------------|-------------------------------------------|
| visitante_evaluador.py          | # Lógica de evaluación (Visitor)          |
| manejador_errores.py            | # Manejo de errores de sintaxis           |
| ejemplos/                       |                                           |
| operaciones_basicas.rc          |                                           |
| variables.rc                    |                                           |
| precedencia.rc                  |                                           |
| area_circulo.rc                 |                                           |
| ciclofor_in.rc                  |                                           |
| estadisticas.rc                 |                                           |
| prueba_completa.rc              |                                           |
| tablas.rc                       |                                           |
| tests/                          |                                           |
| test_risco.py                   | # Suite de pruebas automatizadas (pytest) |
| generar.sh                      | # Script para regenerar el parser         |
| requisitos.txt                  | # Dependencias Python                     |

## 9.1 Cómo ejecutar RISCO

|                                                         |
|---------------------------------------------------------|
| # 1. Clonar el repositorio                              |
| git clone https://github.com/J-Detective-J/Risco-V1.git |
| cd Risco-V1                                             |
|                                                         |
| # 2. Crear entorno virtual e instalar dependencias      |
| python3 -m venv venv                                    |
| source venv/bin/activate                                |
| pip install -r requisitos.txt                           |
|                                                         |
| # 3. Generar el parser desde la gramática               |
| chmod +x generar.sh && ./generar.sh                     |
|                                                         |
| # 4. Modo interactivo                                   |
| python src/risco.py                                     |
|                                                         |
| # 5. Ejecutar un archivo .rc                            |
| python src/risco.py ejemplos/prueba_completa.rc         |
|                                                         |
| # 6. Correr la suite de pruebas                         |
| pytest tests/ -v                                        |